

Title

Smart Autonomous Vacuum Cleaning Agent For Hospitals

Aim

To design and implement a Smart Autonomous Vacuum Cleaning Agent using Python that can detect dirty locations, move between different locations, clean them, monitor its battery level, automatically move to a charging station when the battery is low, and calculate its performance measure.

Experiment Objectives

To design functionality for a vacuum agent that:

1. Detects clean and dirty locations in an environment.
2. Automatically cleans dirty locations.
3. Moves from one location to another as required.
4. Maintains a performance measure based on its actions.
5. Monitors the battery level of the vacuum cleaner.
6. Moves to a predefined charging station when the battery becomes low.
7. Recharges the battery and continues the cleaning process.
8. Demonstrates the working of a Simple Reflex Agent in a real-world environment such as a hospital.

Theory

An AI agent is a system that perceives its environment through sensors and performs actions using actuators to achieve a particular goal.

A Simple Reflex Agent selects an action based on the current state of the environment using predefined condition-action rules.

For the proposed vacuum cleaning agent:

- 0 represents a Clean location.
- 1 represents a Dirty location.

The environment consists of multiple locations such as:

- Reception
- ICU
- Ward
- Corridor
- Pharmacy
- Waiting Area

The agent randomly starts at one of these locations.

The agent follows rules such as:

- Rule 1: If the current location is dirty, clean it.
- Rule 2: If another location is dirty, move to that location.
- Rule 3: If the battery level is less than or equal to 30%, move to the charging station.
- Rule 4: After charging, resume the cleaning operation.

The performance measure is calculated using rewards and penalties.

Action -> Performance

Cleaning a dirty location -> +10

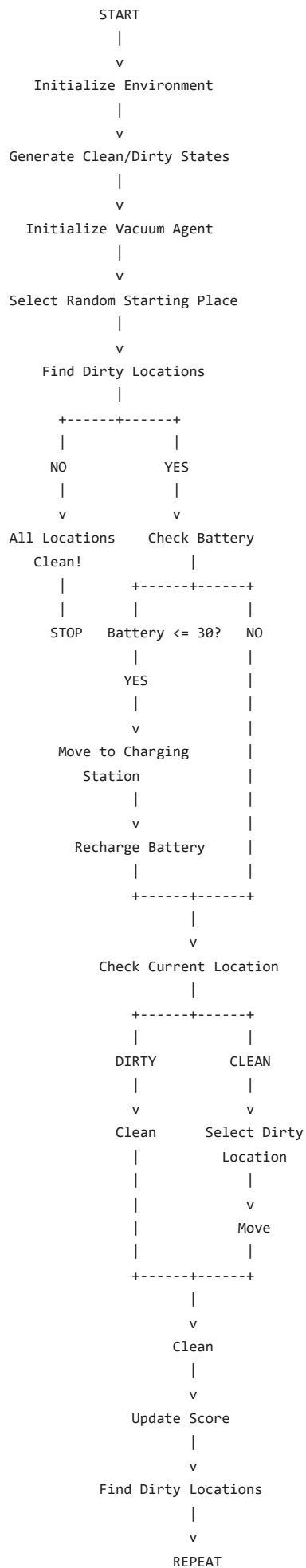
Moving to another location -> -1

Moving to charging station -> -1

Successful charging -> +2

Thus, the agent attempts to clean all locations while minimizing unnecessary movement and managing its battery.

Flowchart Details



Code

```
import random

class Env:
    def __init__(self):
        self.locationConditions={
            "Reception": random.randint(0, 1),
            "ICU": random.randint(0, 1),
            "Ward": random.randint(0, 1),
            "Corridor": random.randint(0, 1),
            "Pharmacy": random.randint(0, 1),
            "Waiting Area": random.randint(0,1)
        }

class vaccumCleaner:
    def __init__(self,env):
        self.env=env
        self.currLocation=random.choice(list(self.env.locationConditions.keys()))

        self.battery=100
        self.score=0
        self.movements=0
        self.chargingCount=0
        print("Initial conditions: ",env.locationConditions)
        print("Vaccum Cleaner starts at :",self.currLocation)
        print("Battery: ",self.battery,"%")
        self.run_cleaner()

    def find_dirty_locations(self):
        dirtyLoc=[]
        for location,condition in self.env.locationConditions.items():
            if(condition==1):
                dirtyLoc.append(location)
        return dirtyLoc

    def move(self,location):
        self.score-=1
        self.battery-=5
        self.movements+=1
        print("\nMoving from ",self.currLocation," to ",location)
        self.currLocation=location
        print("Battery: ",self.battery,"%")

    def clean(self):
        self.score+=10
        self.battery-=20
        print("\nCleaning ",self.currLocation,...)
        self.env.locationConditions[self.currLocation]=0
        print("Cleaning completed")
        print("Battery: ",self.battery,"%")

    def charge(self):
        print("\nBattery is low!")
        print("Moving to charging station")
        self.score-=1
        self.battery-=5
        self.movements+=1

        print("Battery charged successfully!")
        self.score+=2
        self.battery=100
        self.chargingCount+=1
        print("Battery: ",self.battery,"%")

    def final_disp(self):
        print("\n\nFinal Results:")
        print("Environment: ",self.env.locationConditions)
        print("Current position: ",self.currLocation)
        print("Battery: ",self.battery)
        print("Performance measure: ",self.score)
        print("Number of movements: ",self.movements)
        print("Number of charging operations: ",self.chargingCount)

    def run_cleaner(self):
        if(self.env.locationConditions[self.currLocation]==1):
            self.clean()
        dirtyLocations=self.find_dirty_locations()
        if(len(dirtyLocations)==0):
```

```
        print("\nAll locations are clean!")
    else:
        for loc in dirtyLocations:
            if(self.battery<=30):
                self.charge()
            if(self.currLocation==loc):
                continue
            else:
                self.move(loc)
                self.clean()
        self.final_disp()
```

```
Testenv=Env()
TestVaccumCleaner=vaccumCleaner(Testenv)
```

Initial conditions: {'Reception': 1, 'ICU': 1, 'Ward': 0, 'Corridor': 1, 'Pharmacy': 1, 'Waiting Area': 0}
Vaccum Cleaner starts at : ICU
Battery: 100 %

Cleaning ICU ...
Cleaning completed
Battery: 80 %

Moving from ICU to Reception
Battery: 75 %

Cleaning Reception ...
Cleaning completed
Battery: 55 %

Moving from Reception to Corridor
Battery: 50 %

Cleaning Corridor ...
Cleaning completed
Battery: 30 %

Battery is low!
Moving to charging station
Battery charged successfully!
Battery: 100 %

Moving from Corridor to Pharmacy
Battery: 95 %

Cleaning Pharmacy ...
Cleaning completed
Battery: 75 %

Final Results:
Environment: {'Reception': 0, 'ICU': 0, 'Ward': 0, 'Corridor': 0, 'Pharmacy': 0, 'Waiting Area': 0}
Current position: Pharmacy
Battery: 75
Performance measure: 38
Number of movements: 4
Number of charging operations: 1

Experiment Outcome

The experiment demonstrates the implementation of a Simple Reflex Vacuum Agent using Python.

The agent is able to:

- Detect dirty locations.
- Clean dirty locations.
- Move between locations.
- Monitor its battery.
- Charge when the battery is low.
- Maintain a performance score.
- Display the final cleaning status and performance statistics.

Result Analysis

The Smart Autonomous Vacuum Cleaning Agent was successfully implemented.

The agent randomly generated the cleanliness conditions of different hospital locations and selected a random starting location. It successfully identified dirty locations and cleaned them.

The agent also maintained a performance score. Cleaning a location increased the score, while movement and charging introduced a small penalty. The battery level was continuously monitored, and when it reached the specified threshold, the agent performed the charging operation before continuing its cleaning task.

Conclusion

The Smart Autonomous Vacuum Cleaning Agent was successfully designed and implemented using Python. The agent demonstrates the concept of a Simple Reflex Agent by taking actions based on the current condition of its environment.

The proposed system extends the traditional two-location vacuum problem by introducing multiple locations, autonomous movement, battery monitoring, charging functionality, and performance measurement. The same concept can be further extended to real-world applications such as hospital cleaning, college cleaning, office cleaning, and parking-area cleaning.

The experiment provides a practical understanding of how AI agents perceive an environment, make decisions based on predefined rules, perform actions, and evaluate their performance.

Github link:

[click](#)

Colab Link

[click](#)